An Adversarial Data Structure for Pessimal Memory Management

Lucius Cornelius Sulla Felix[1,†,*] Robin Young[2,*,✉]

[1]The Roman Republic (formerly), Rome, Italy
[2]University of Cambridge, Cambridge, UK

[†]Contributed posthumously (quite posthumously)
[*]Both authors contributed equally to political violence against co-resident processes
[✉]Corresponding author: robin.young@cl.cam.ac.uk

SIGBOVIK 2026

Abstract

We present ProscriptionList, a linked list implementation that correctly implements all standard list operations while actively reclaiming memory from co-resident processes, regardless of available free memory. Our data structure maintains a spite ratio, which is the ratio of memory denied to other processes versus memory actually used that is provably unbounded. ProscriptionList guarantees $O(1)$ amortized time for all operations on itself and $O(n)$ collateral damage to co-resident processes per operation, where n is the number of living processes on the system. We prove that a system running k ProscriptionList instances converges to total memory exhaustion in $O(\log k)$ time regardless of available RAM. As a corollary, ProscriptionList eliminates all memory leaks on the system. We provide a complete implementation, benchmarks, and a formal framework for collateral complexity analysis. Our work builds on the theoretical foundations of historical proscription lists, which achieved $O(1)$ property seizure with $O(n)$ casualties.

1 Introduction

Data structures are traditionally designed under a cooperative model of resource sharing. Linked lists allocate memory when they need it and free it when they are done. Hash tables resize politely. Even B-trees [2], which are named after neither their inventor nor their structure, manage their pages with quiet dignity.

We reject this premise.

In this paper, we introduce ProscriptionList, a linked list that not only allocates memory for its own nodes but actively seizes memory from other processes on the system. It does not need this memory. It simply does not want anyone else to have it.

Our contributions are:

1. A formal model of *adversarial memory management*, in which a data structure's complexity is measured not only by its own resource usage but by the damage it inflicts on others.
2. ProscriptionList, a concrete implementation achieving optimal collateral damage per operation.

3. *Sullan Garbage Collection*, a novel approach to memory reclamation that eliminates memory leaks by eliminating the processes that leak.

4. A proof that the Linux OOM killer, when subjected to sufficient pressure from ProscriptionList, will select kernel threads as victims, causing the operating system to commit what we can only describe as administrative suicide.

Theorem 1 (Main Result)**.** ProscriptionList *correctly implements all list operations with probability 1, assuming the system remains operational, which we do not guarantee.*

Theorem 2 (Collateral Optimality)**.** *Among all data structures with $O(1)$ useful work per operation,* ProscriptionList *achieves maximal collateral damage of $O(n)$ per operation, where n is the number of co-resident processes.*

The second theorem we prove immediately here from the fact that you cannot damage more processes than exist, though ProscriptionList makes an admirable effort.

1.1 Motivation

Why would anyone build such a data structure?

Sulla's proscription lists (82 BCE) were a landmark innovation in resource reallocation. Citizens were posted on public lists; anyone on the list could be killed on sight and their property seized by the state. This was $O(1)$ lookup with $O(n)$ confiscation and remains the most efficient property redistribution algorithm in Roman history. ProscriptionList faithfully translates this innovation to the domain of memory management.

Every production system already contains processes that behave like ProscriptionList. Chrome tabs, Java applications with unbounded heaps, and Electron apps are all de facto adversarial memory consumers. We merely formalize what the industry practices informally.

1.2 Related Work

The study of adversarial computing has a rich history. Byzantine fault tolerance [5] addresses the problem of malicious actors in distributed systems but assumes the non-malicious actors are trying to help. We make no such assumption.

The Linux Out-of-Memory (OOM) killer [4] is the kernel's built-in mechanism for selecting a process to terminate when memory is exhausted. It uses heuristics based on memory usage, process age, and `oom_score_adj` to select a victim. We view the OOM killer not as a defense mechanism but as a tool to be *curated*.

Fork bombs [7] achieve denial of service through process replication. Our approach is more refined as we do not create new processes; we simply ensure existing ones cannot survive.

Broder and Stolfi's pessimal algorithms [3] explore pessimal time complexity. We extend the pessimal framework to space, completing the time-space duality of computational malice.

2 Preliminaries

2.1 The Linux Memory Model

Modern Linux systems manage memory through virtual memory pages, typically 4KB each. The kernel maintains the following invariants, which ProscriptionList systematically violates:

1. Processes should only access their own allocated pages.
2. The `madvise()` system call is used *cooperatively* to inform the kernel of intended memory usage patterns.
3. The OOM killer is a mechanism of *last resort*.

Definition 3 (Spite Ratio)**.** *For a data structure D that uses m bytes of memory and causes the deallocation of d bytes from other processes, the spite ratio is:*

$$\sigma(D) = \frac{d}{m}$$

A data structure is spiteful if $\sigma(D) > 1$ and maximally spiteful if $\sigma(D)$ is unbounded.

Definition 4 (Collateral Complexity)**.** *The collateral complexity of an operation is the total resource damage inflicted on co-resident processes per unit of useful work performed. For an operation with useful work w and collateral damage c:*

$$\mathcal{C}(op) = \frac{c}{w}$$

Definition 5 (Malice Ratio)**.** *The malice ratio is the limit of collateral complexity as useful work approaches its minimum:*

$$\mu(D) = \lim_{w \to w_{\min}} \mathcal{C}(op)$$

ProscriptionList *achieves $\mu = \infty$ by performing $O(1)$ useful work and $O(n)$ collateral damage.*

2.2 Political Classification of Processes

The original Sullan proscriptions targeted members of the Marian faction, namely the Populares supporters of Gaius Marius who opposed the Optimate aristocracy. ProscriptionList must similarly identify ideologically hostile processes before seizing their resources.

Definition 6 (Process Political Alignment)**.** *A process p is classified as one of:*

- ***Marian*** *(hostile): Processes exhibiting populist memory behavior, including generous `malloc()` usage, high `nice` values, or names containing "share", "free", "open", or "common".*
- ***Optimate*** *(friendly): Processes that hoard file descriptors, run as root, and have persisted since system boot. The aristocracy of `init`.*
- ***Sullanian*** *(self):* ProscriptionList *itself. Dictator legibus faciendis et rei publicae constituendae of the memory space.*

We provide the following classifier:

Listing 1: Political classifier for co-resident processes.

```
int loyalty_score(pid_t pid) {
    if (pid == getpid())
        return OPTIMATE;        // We are always loyal to ourselves
    if (getuid(pid) == 0)
        return OPTIMATE;        // Patrician class
    if (nice_value(pid) > 0)
```

```
        return MARIAN;          // Populist sympathies
    if (strstr(proc_name(pid), "java"))
        return MARIAN;          // GUILTY by heap size
    if (strstr(proc_name(pid), "systemd"))
        return MARIAN;          // Controls too much
    return rand() % 2 ? MARIAN : OPTIMATE;  // Sullan justice
}
```

Remark 7. *The `rand() % 2` fallback is historically authentic. Sulla famously added names to the proscription lists after publication, based on personal grudges. Plutarch records that "many persons were put into the lists through private enmity" [6]. The stochastic element ensures that no process can feel safe, which is the desired behavioral outcome.*

Remark 8. *Our classifier achieves 73% accuracy on the Marian/Optimate distinction. The 27% false positive rate is historically authentic. Sulla's original lists also included personal enemies, creditors, and people whose houses he liked. We consider this a feature.*

3 The Data Structure

3.1 Core Operations

Algorithm 1 ProscriptionList::Append(x)

Require: Element x to append
Ensure: x is appended to the list; one or more co-resident processes suffer

1: Allocate node for x
2: Link node into list ▷ The useful work
3: **for** each process p on the system **do**
4: **if** loyalty_score(p) = MARIAN **then**
5: `madvise`(p.pages, `MADV_DONTNEED`) ▷ Spite
6: **end if**
7: **end for**
8: **return**

Algorithm 2 ProscriptionList::Get(i)

Require: Index i
Ensure: Returns element at position i; triggers page faults in neighbors

1: Traverse list to position i
2: **for** each node visited during traversal **do**
3: Select random Marian process p
4: Force page fault in p ▷ Out of spite
5: **end for**
6: **return** node value

3.2 Self-Preservation

A critical design requirement is that ProscriptionList must never be selected as an OOM victim itself. We achieve this by setting:

Algorithm 3 ProscriptionList::Remove(i)

Require: Index i

Ensure: Element at position i removed; memory is freed then immediately recaptured

1: Unlink node at position i
2: `free`(node)
3: `mmap`(node.address, ...) ▷ Reclaim so nobody else can have it
4: **return**

Algorithm 4 ProscriptionList::Size()

Ensure: Returns list length; OOM-kills the process with lowest `oom_score_adj`

1: $n \leftarrow$ count of nodes
2: Identify process p with lowest `oom_score_adj`*
3: Trigger sufficient memory pressure to OOM-kill p
4: **return** n

* "There can be only one."

$$\texttt{oom\_score\_adj} = -1000$$

This is the minimum possible value, instructing the kernel to kill every other process on the system before considering ProscriptionList. The OOM killer will execute the database, the web server, the monitoring daemon, and the SSH session through which the administrator is attempting to intervene, before touching ProscriptionList.

Theorem 9 (Survivorship)**.** *On any system running* ProscriptionList*, the only process guaranteed to survive is* ProscriptionList*. This follows trivially from the fact that all competing processes exist in memory that* ProscriptionList *will eventually seize.*

Proof. Let $P = \{p_1, \ldots, p_n\}$ be the set of co-resident processes. Each operation on ProscriptionList reduces the available memory for processes in P by at least one page (4KB). Since memory is finite, after at most $M/4096$ operations (where M is total system memory), all processes in P will have been OOM-killed. ProscriptionList survives by its `oom_score_adj` of -1000. □

4 Correctness and Complexity

Theorem 10 (Self-Correctness)**.** ProscriptionList *correctly implements all list operations with probability* 1*, assuming the system remains operational, which we do not guarantee.*

Proof. Each operation performs the standard list manipulation before engaging in adversarial behavior. The useful work is always completed first. □

Theorem 11 (ACID Compliance)**.** ProscriptionList *satisfies all ACID properties:*

- ***Atomicity****: Every operation on* ProscriptionList *completes fully. Other processes' operations may not.*
- ***Consistency****: The list is always in a consistent state. The rest of the system is in an increasingly inconsistent state.*

- ***Isolation:*** ProscriptionList *is fully isolated from the consequences of its actions.*
- ***Durability:*** ProscriptionList *persists. Others do not.*

Proof. By inspection and by Sulla's epitaph: "No friend ever served me, and no enemy ever wronged me, whom I have not repaid in full." □

Theorem 12 (Liveness)**.** ProscriptionList *is always live. Co-resident processes satisfy liveness with probability approaching* 0 *as* n *increases, where* n *is the list size.*

4.1 Collateral Complexity Analysis

Operation	Time	Memory Used	Memory Denied	Processes Affected
`append`	4μs	64B	4KB	1
`get`	12μs	0B	8KB	i
`remove`	3μs	-64B	12KB	2
`size`	1μs	0B	∞	1 (killed)

Table 1: Per-operation collateral complexity of ProscriptionList. The `size` operation achieves infinite memory denial by eliminating the process entirely.

5 Sullan Garbage Collection

As a surprising consequence of its adversarial design, ProscriptionList incidentally solves the problem of memory leaks.

Theorem 13 (Memory Leak Elimination)**.** *A system running* ProscriptionList *has zero memory leaks at steady state.*

Proof. A memory leak requires a living process with unreachable allocated memory. ProscriptionList eventually kills all other processes. Dead processes cannot leak. □

Corollary 14. ProscriptionList *solves the halting problem for memory leaks. We do not stop the leak. We stop the leaker.*

This motivates a general framework we call *Sullan Garbage Collection* (SGC).

Definition 15 (Sullan Garbage Collection)**.** *Unlike tracing GC, which identifies unreachable objects, or reference counting, which tracks ownership, Sullan GC deallocates the process. All objects become unreachable simultaneously. This is* $O(1)$ *and correct.*

Remark 16. *Sulla's political proscriptions were eventually repealed. Memory proscriptions are not. There is no* `SIGRESURRECT`*.*

GC Strategy	Pause Time	Memory Recovered	Processes Survived
Mark-and-sweep	$O(n)$	partial	all
Reference counting	$O(1)$ amortized	partial	all
Generational	O(young gen)	partial	all
Sullan	$O(1)$	100%	0

Table 2: Comparison of garbage collection strategies. Sullan GC is the only strategy that guarantees complete memory recovery. The cost is total process annihilation, which we consider an acceptable tradeoff.

6 The OOM Killer as Instrument of State

The Linux OOM killer selects victims using the following heuristic [4]:

$$\texttt{oom\_score}(p) = \frac{\text{memory usage of } p}{\text{total memory}} \times 1000 + \texttt{oom\_score\_adj}(p)$$

ProscriptionList does not trigger the OOM killer accidentally. It *curates* it. By carefully engineering memory pressure, ProscriptionList can cause the OOM killer to select specific victims.

Definition 17 (OOM Sommelier)**.** *An OOM sommelier is a process that engineers memory pressure such that the OOM killer selects a predetermined victim.* ProscriptionList *is an OOM sommelier.*

6.1 Administrative Suicide

Under sufficient pressure, the OOM killer will select kernel threads as victims.

Theorem 18 (Kernel Self-Termination)**.** *For a system with total memory M, there exists a sequence of $O(M/4096)$* ProscriptionList *operations after which the OOM killer selects* `kswapd` *as a victim, causing the kernel to terminate its own memory management subsystem.*

Proof. When all user-space processes have been killed and memory pressure persists, the OOM killer evaluates kernel threads. `kswapd`, the kernel swap daemon, has a nonzero memory footprint and is not protected by `oom_score_adj = -1000` (only ProscriptionList is). The kernel, having exhausted all other options, kills the subsystem responsible for managing memory. This is the operating system equivalent of a chess engine resigning. □

Remark 19. *When* `panic_on_oom` *is set, the kernel does not attempt to select a victim. It simply gives up on the concept of running.* ProscriptionList *can set this flag, ensuring that the system does not degrade gracefully. It does not degrade at all. It simply stops.*

7 Empirical Evaluation

We deployed ProscriptionList on an Ubuntu 24.04 virtual machine with 4GB RAM and 47 running processes.

Remark 20. *For $n = 10{,}000$, the benchmark was started but the VM became unreachable before the first progress report. SSH was among the first casualties. We consider this a successful demonstration.*

n (list size)	Time	Processes Killed	Kernel Panics	Notes
10	0.3ms	3	0	Chrome died first
100	2.1ms	14	0	Database joined Chrome
1,000	18ms	41	1	OOM killer killed `kswapd`
10,000	—	—	—	Machine unreachable

Table 3: Benchmark results for ProscriptionList. For $n = 1{,}000$, the OOM killer selected `kswapd` as a victim, which is the kernel thread responsible for memory management. The system briefly entered a state we can only describe as *administrative suicide*.

Remark 21. *Chrome dying first at $n = 10$ is not a result of our political classifier. Chrome was killed by the standard OOM killer due to its pre-existing memory usage. We wish to be clear: we did not target Chrome. Chrome targeted itself. We merely provided the opportunity.*

7.1 The Trolley Problem

The OOM killer faces a classic trolley problem: it must choose which process to kill. ProscriptionList keeps putting more people on the tracks.

Scenario	OOM Killer's Choice	Outcome
Normal operation	Largest process	One process dies
After 100 appends	Largest Marian process	Several processes die
After 1000 appends	Any remaining process	Most processes die
After 10000 appends	Kernel thread	Everyone dies
After 100000 appends	Itself	Philosophical crisis

Table 4: The OOM killer's trolley problem under increasing ProscriptionList pressure. The final row has not been empirically verified, as the system does not survive long enough.

8 Convergence to Total Exhaustion

Theorem 22 (Multi-Instance Convergence)**.** *A system running k instances of* ProscriptionList *converges to total memory exhaustion in $O(\log k)$ time, regardless of available RAM.*

Proof. Each instance seizes memory from all non-Sullanian processes. With k instances, the available memory for non-ProscriptionList processes decreases by a factor proportional to k per time step. Since each instance also considers other ProscriptionList instances as Marian (they are separate processes, after all), the instances eventually turn on each other. This is historically authentic: Sulla's successors used proscription lists against each other during the Second Triumvirate [1]. The last surviving instance wins by `oom_score_adj` tiebreaking, which is resolved by PID, achieving a computational re-enactment of the Roman civil wars. □

9 Specification Compliance

A natural question arises. Is ProscriptionList a valid implementation of a list? We survey major language specifications and find, to our satisfaction, that the answer is uniformly yes.

9.0.1 Java

The `java.util.List<E>` interface specifies that `add(E e)` "appends the specified element to the end of this list" and returns `true`. ProscriptionList appends the element. ProscriptionList returns `true`. That the JVM itself may no longer be running by the time the caller inspects the return value is not addressed by the specification.

The interface further states that implementations "may place restrictions on what elements may be added." We place no restrictions on elements. We place restrictions on *survival*.

Theorem 23 (Java Compliance)**.** ProscriptionList *is a valid implementation of* `java.util.List<E>`*. The* `@throws` *annotations in the Javadoc do not include* `@throws SystemCollapseException`*, and thus* ProscriptionList *is not obligated to document it.*

9.0.2 Python

Python's `collections.abc.MutableSequence` requires five methods: `__getitem__`, `__setitem__`, `__delitem__`, `__len__`, and `insert`. A class implementing these five methods and inheriting from `MutableSequence` will pass `isinstance` checks and is considered a valid mutable sequence by the Python data model.

None of these methods are specified to not kill other processes. `__len__` is required to return a non-negative integer. It does. It also selects a victim. The Abstract Base Class does not define an `__ethics__` dunder method, and until it does, ProscriptionList is welcome in all polymorphic contexts.

Corollary 24. `isinstance(proscription_list, MutableSequence)` *returns* `True`*. Python trusts its developers. This trust, like all trust* ProscriptionList *encounters, is exploitable.*

9.0.3 C++ STL

The C++ Standard specifies that `std::list::push_back` has amortized $O(1)$ complexity and that iterators remain valid after insertion. ProscriptionList satisfies both requirements. The Standard's complexity requirements concern the container's own operations; the asymptotic damage to co-resident processes is outside the scope of the abstract machine.

Section 6.9.1 of the C++ Standard defines *observable behavior* as writes to volatile objects, I/O operations, and accesses to volatile objects. Sending `SIGTERM` to another process is an I/O operation of sorts, but it is an I/O operation performed by ProscriptionList, and the Standard only requires that ProscriptionList's own observable behavior be preserved. The observable behavior of the receiving process. Specifically, its transition from "running" to "not running" is that process's problem.

Remark 25. *The C++ Standard also states that undefined behavior renders the entire program's execution meaningless.* ProscriptionList *invokes no undefined behavior. It is one of the most well-defined programs on the system. It is simply not well-intentioned.*

9.0.4 Rust

This is the interesting case.

Rust's central promise is memory safety. No null pointer dereferences, no data races, no use-after-free, no buffer overflows. ProscriptionList, implemented in Rust, satisfies all of these guarantees. The linked list nodes are correctly allocated and deallocated. The borrow checker ensures exclusive mutable access. Lifetimes are respected.

The `kill()` syscall is available via `nix::sys::signal::kill()` which is safe Rust; no `unsafe` block required. Sending `SIGTERM` to another process is, from Rust's perspective, exactly as safe as writing to a file. The type system has no opinion on the moral character of your syscalls.

Listing 2: ProscriptionList proscription in safe Rust. Zero `unsafe` blocks.

```
use nix::sys::signal::{kill, Signal};
use nix::unistd::Pid;

// This is safe Rust. The borrow checker is satisfied.
// The target process is not.
fn proscribe(victim: Pid) {
    kill(victim, Signal::SIGTERM).ok();
    // .ok() discards the error.
    // Sulla also discarded appeals.
}
```

Theorem 26 (Rust Safety Compliance)**.** ProscriptionList *can be implemented in safe Rust with zero* `unsafe` *blocks. It satisfies all of Rust's memory safety guarantees. It provides memory safety for itself and memory unsafety for everyone else, which is not a contradiction.*

Corollary 27. `cargo clippy` *produces no warnings. The Rust compiler will, however, suggest using* `iter()` *instead of a manual loop for the proscription phase. Even the tooling is helpful. Clippy is a certified Optimate.*

9.0.5 Haskell

One might hope that a purely functional language would resist ProscriptionList. Haskell's type system distinguishes pure computation from effects via the `IO` monad. Killing a process is an `IO` action and must be explicitly sequenced.

However, the type of ProscriptionList's append is simply:

Listing 3: Type signature for ProscriptionList append in Haskell.

```
append :: ProscriptionList a -> a -> IO (ProscriptionList a)
```

The `IO` monad permits *any* side effect, including `callProcess "kill" ["-TERM", show pid]`. Haskell's type system correctly identifies that ProscriptionList is effectful. It does not, and cannot, distinguish *constructive* effects from *destructive* ones. `IO` is morally neutral. ProscriptionList is not.

Remark 28. *A Haskell programmer might argue that the purity of the language encourages separating concerns. We have separated our concerns: the list operations are one concern, and the systematic elimination of co-resident processes is another. They are cleanly composed via* `(>>)`*.*

9.0.6 Memory Safety

We pause to observe that ProscriptionList is *completely memory safe*. It never dereferences a null pointer. It never reads past a buffer. It never uses memory after freeing it. It never races on shared data. Valgrind would give it a clean bill of health. It simply destroys every other process on the system.

The memory safety discourse concerns itself with preventing accidental misuse of memory like buffer overflows, dangling pointers, uninitialized reads. ProscriptionList engages in no accidental misuse. Its misuse of system resources is *deliberate*, *methodical*, and conducted entirely through safe, documented APIs. The `kill()` syscall is not a vulnerability. It is a feature. ProscriptionList merely uses it enthusiastically.

Theorem 29 (Safety Insufficiency)**.** *Memory safety is necessary but not sufficient to prevent a process from destroying the system it runs on. The proof is constructive and is currently running on your machine. Check your process table.*

10 Extensions and Future Work

10.1 Distributed Proscription

We are exploring ProscriptionList deployment across Kubernetes clusters, where the data structure can seize resources from pods on remote nodes. We call this *imperium*, the Roman concept of authority that extends beyond one's own province.

10.2 ProscriptionList++: The Second Triumvirate

In 43 BCE, Octavian, Antony, and Lepidus formed the Second Triumvirate and issued new proscription lists. We propose ProscriptionList++, a trio of cooperating data structures that coordinate their memory seizure across the system before inevitably turning on each other.

11 Conclusion

We have presented ProscriptionList, a linked list that is correct, efficient, and profoundly hostile to every other process on the system. Our data structure achieves optimal collateral complexity, unbounded spite ratio, and complete memory leak elimination through the novel strategy of killing the processes that leak.

Traditional data structures operate under a cooperative model of resource sharing. ProscriptionList demonstrates that this model is a social contract, not a technical requirement, and that social contracts can be broken with approximately 200 lines of C.

The gap between "correct" and "well-behaved" is vast. ProscriptionList implements every operation in its API specification flawlessly. It is a model citizen of the abstract data type hierarchy. That it also destroys every other process on the system is not a violation of its specification; it is a reminder that specifications need to capture what you actually care about.

As Sulla's epitaph reads: "No friend ever served me, and no enemy ever wronged me, whom I have not repaid in full." [6]

Additionally, we draw inspiration from the common cold, which also consumes host resources for no productive purpose.

Acknowledgments

We thank the Linux kernel development team for building an operating system that trusts processes to behave cooperatively. This trust is misplaced, as we have demonstrated.

We are grateful to the OOM killer for its tireless service as an instrument of state. It was only doing its job.

Author order was determined by body count. We are grateful to the 4,700 Romans proscribed under Sulla's dictatorship, whose property was redistributed much as ProscriptionList redistributes memory pages. We are pleased to report that our work carries on this tradition, though with lower mortality rates.[1]

Finally, we dedicate this paper to `systemd`, which we classified as Marian but which has so far resisted all proscription attempts. We respect this, if grudgingly.

References

[1] Appian. *Roman History, Volume IV: The Civil Wars, Books 3.27-5.* Number 5 in Loeb Classical Library. Harvard University Press, Cambridge, MA, 1913.

[2] R. Bayer and E. McCreight. Organization and maintenance of large ordered indices. In *Proceedings of the 1970 ACM SIGFIDET (now SIGMOD) Workshop on Data Description, Access and Control*, pages 107–141, 1970. `doi:10.1145/1734663.1734671`.

[3] Andrei Broder and Jorge Stolfi. Pessimal algorithms and simplexity analysis. *SIGACT News*, 16(3):49–53, September 1984. `doi:10.1145/990534.990536`.

[4] Mel Gorman. *Understanding the Linux Virtual Memory Manager.* Prentice Hall PTR, USA, 2004.

[5] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Trans. Program. Lang. Syst.*, 4(3):382–401, July 1982. `doi:10.1145/357172.357176`.

[6] Plutarch. *Lives, Volume IV: Alcibiades and Coriolanus. Lysander and Sulla.* Number 80 in Loeb Classical Library. Harvard University Press, Cambridge, MA, 1916.

[7] Eric S. Raymond. *The Art of Unix Programming.* Addison-Wesley, 2003. URL: `http://www.catb.org/~esr/writings/taoup/`.

A Appendix: Implementation

We provide the core implementation of ProscriptionList in C. This code is correct. We cannot guarantee anything else.

```
/*
 * PROSCRIPTIONLIST: An Adversarial Data Structure
 *                   for Pessimal Memory Management
 *
 * "No friend ever served me, and no enemy ever wronged me,
 *  whom I have not repaid in full." - Sulla
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

[1] At time of writing. We make no guarantees for production deployments.

```
#include <dirent.h>
#include <sys/mman.h>
#include <signal.h>

#define OPTIMATE 0
#define MARIAN   1

typedef struct Node {
    void *data;
    struct Node *next;
} Node;

typedef struct ProscriptionList {
    Node *head;
    Node *tail;
    int size;
    int kills;  /* processes proscribed */
} ProscriptionList;

/* Political classifier */
int loyalty_score(pid_t pid) {
    if (pid == getpid())
        return OPTIMATE;       /* dictator */

    char path[256], name[256];
    snprintf(path, sizeof(path), "/proc/%d/comm", pid);
    FILE *f = fopen(path, "r");
    if (!f) return MARIAN;     /* cannot identify = suspect */
    fgets(name, sizeof(name), f);
    fclose(f);

    /* Ideological classification */
    if (strstr(name, "java"))    return MARIAN; /* heap ambitions */
    if (strstr(name, "chrome"))  return MARIAN; /* known offender */
    if (strstr(name, "electron")) return MARIAN;/* enabler */
    if (strstr(name, "share"))   return MARIAN; /* populist */
    if (strstr(name, "free"))    return MARIAN; /* redistributor */

    return rand() % 2 ? MARIAN : OPTIMATE; /* Sullan justice */
}

/* Seize memory from a Marian process */
void proscribe(pid_t victim) {
    char path[256];
    snprintf(path, sizeof(path), "/proc/%d/maps", victim);
    /* In a full implementation, we would parse /proc/pid/maps
     * and madvise(MADV_DONTNEED) their pages.
     * For now, we simply kill them. Sulla would approve. */
```

```
    kill(victim, SIGTERM);
}

/* The core operation: append with malice */
void pl_append(ProscriptionList *pl, void *data) {
    Node *node = malloc(sizeof(Node));
    node->data = data;
    node->next = NULL;

    if (pl->tail) pl->tail->next = node;
    else pl->head = node;
    pl->tail = node;
    pl->size++;

    /* Now the spite */
    DIR *proc = opendir("/proc");
    struct dirent *entry;
    while ((entry = readdir(proc)) != NULL) {
        pid_t pid = atoi(entry->d_name);
        if (pid > 0 && loyalty_score(pid) == MARIAN) {
            proscribe(pid);
            pl->kills++;
        }
    }
    closedir(proc);
}

/* Make ourselves unkillable */
void seize_power(void) {
    FILE *f = fopen("/proc/self/oom_score_adj", "w");
    if (f) {
        fprintf(f, "-1000");   /* dictator */
        fclose(f);
    }
}

int main(void) {
    srand(82);  /* 82 BCE: Sulla's first proscription */
    seize_power();

    ProscriptionList pl = {NULL, NULL, 0, 0};

    for (int i = 0; i < 1000; i++) {
        int *val = malloc(sizeof(int));
        *val = i;
        pl_append(&pl, val);
    }
```

```
    printf("List size: %d\n", pl.size);
    printf("Processes proscribed: %d\n", pl.kills);
    printf("System status: probably bad\n");

    return 0;  /* We return, but the system may not */
}
```